

Analiza efektywności Algorytmu Genetycznego (Genetic Algorithm) w rozwiązywaniu Problemu Komiwojażera (TSP)

Kacper Piekut

Numer albumu: 280879

Wydział: W4N, Rok akademicki: 2025/2026, Semestr: 6

Politechnika Wrocławska

Streszczenie—W niniejszym sprawozdaniu przedstawiono analizę populacyjnego algorytmu metaheurystycznego - Algorytmu Genetycznego (GA) - dedykowanego do rozwiązywania problemu komiwojażera w wersji symetrycznej (TSP). Zbadano wpływ kluczowych parametrów: rozmiaru populacji, prawdopodobieństwa mutacji, liczby pokoleń, metod mutacji (Swap, Insert, Invert) oraz operatora krzyżowania Order Crossover (OX). Proces badawczy oparto na izolacji zmiennych konfiguracyjnych w celu określenia ich wpływu na czas obliczeń oraz błąd względny uzyskiwanych tras.

Index Terms—Problem Komiwojażera, TSP, Algorytm Genetyczny, GA, Krzyżowanie OX, Mutacja, Repetitive Nearest Neighbour, RNN.

I. WSTĘP

A. Hipotezy badawcze

Przed przystąpieniem do pomiarów sformułowano następujące hipotezy badawcze:

- **H1 (Skalowalność populacyjna):** Złożoność czasowa pojedynczego pokolenia algorytmu genetycznego wykazuje zależność wielomianową względem rozmiaru instancji $O(N^2)$, co pozwoli na przeszukiwanie instancji rzędu $N = 2500$ w akceptowalnym czasie, niedostępnym dla metod dokładnych.
- **H2 (Wpływ rozmiaru populacji na jakość rozwiązania):** Zwiększenie rozmiaru populacji prowadzi do poprawy jakości rozwiązań końcowych, jednak kosztem wydłużenia czasu obliczeń.
- **H3 (Wpływ prawdopodobieństwa mutacji na różnicowanie populacji):** Wyższe prawdopodobieństwo mutacji zwiększa różnicowanie populacji, co może zapobiegać przedwczesnej zbieżności.
- **H4 (Wpływ liczby pokoleń na jakość rozwiązania):** Zwiększenie liczby pokoleń pozwala na uzyskanie lepszych rozwiązań, jednak po pewnym progu dalsze zwiększanie liczby pokoleń nie przynosi istotnej poprawy.
- **H5 (Przewaga operatora mutacji typu Invert nad pozostałymi metodami):** Operator mutacji Invert wykazuje znacznie wyższą efektywność w przeszukiwaniu przestrzeni rozwiązań niż operatory Swap oraz Insert.

II. METODOLOGIA BADAŃ

A. Środowisko testowe i dane wejściowe

Badania przeprowadzono na laptopie ASUS TUF Gaming F15 (procesor Intel Core i5-11400H 2.7-4.5 GHz, 16 GB RAM DDR4, dysk SSD NVMe 512 GB) pod kontrolą systemu Windows 11 Home (64-bit). Środowiskiem programistycznym był język C++ (kompilator MinGW GCC). W badaniach wykorzystano instancje z biblioteki TSPLIB, dzielące się na:

- **Grafy symetryczne (TSP) instancje:** burma14, gr24, gr48, st70, kroA100, pr124, si175, gr229, lin318, pcb442, rat783, pr1002, rl1304, rl1889, u2319.

B. Opis zaimplementowanego algorytmu

W celu optymalizacji i poszukiwania przybliżonych rozwiązań cykli Hamiltona w strukturach grafowych TSP zaimplementowano heurystykę populacyjną - Algorytm Genetyczny. Algorytm ten przeszukuje przestrzeń stanów wielopunktowo, operując jednocześnie na zbiorze zakodowanych rozwiązań dopuszczalnych (populacji chromosomów).

W każdym pokoleniu, w celu wyłonienia par rodzicielskich do reprodukcji, stosowana jest selekcja turniejowa. Z populacji wybierana jest losowo podgrupa o stałym rozmiarze turnieju $k = 5$, z której osobnik o najniższym koszcie trasy (najwyższym przystosowaniu) wygrywa prawo do reprodukcji. Zapobiega to przedwczesnemu zdominowaniu populacji przez super-osobniki i pozwala kontrolować presję selekcyjną.

Do generowania potomstwa wykorzystano operator krzyżowania dla problemów sekwencyjnych - Order Crossover (OX). Operator ten polega na wylosowaniu dwóch punktów podziału, skopiowaniu wyciętego bloku genów bezpośrednio z pierwszego rodzica do potomka, a następnie uzupełnieniu pozostałych pozycji miastami z drugiego rodzica z zachowaniem ich względnej kolejności, co gwarantuje zachowanie cechy dopuszczalności rozwiązania.

Potomkowie poddawani są operacji mutacji z prawdopodobieństwem przy użyciu jednej z trzech metod (swap, insert, invert).

Algorithm 1 Genetic Algorithm

Require: Graf G , P_{size} , P_m , Gen_{max} , $Tour_{rnn}$, $Cost_{rnn}$

```
1:  $Populacja \leftarrow Populacje(P_{size}, Tour_{rnn}, Cost_{rnn})$ 
2:  $Best \leftarrow ZnajdzNajlepszego(Populacja)$ 
3:  $Stagnacja \leftarrow 0$ 
4: while  $g < Gen_{max}$  and !Timeout and  $Best.cost > OPT$ 
   do
5:   if  $Stagnacja \geq 0.2 \cdot Gen_{max}$  then
6:     break {Kryterium stagnacji}
7:   end if
8:    $NPopulacja \leftarrow \{Best\}$  {Mechanizm elityzmu}
9:   while  $|NPopulacja| < P_{size}$  do
10:     $R_1 \leftarrow SelekcjaTurniejowa(Populacja, 5)$ 
11:     $R_2 \leftarrow SelekcjaTurniejowa(Populacja, 5)$ 
12:     $Potomek \leftarrow OrderCrossover(R_1, R_2)$ 
13:     $Mutuj(Potomek, P_m, Metoda)$ 
14:     $Potomek.cost \leftarrow ObliczKoszt(Potomek)$ 
15:     $NPopulacja \leftarrow NPopulacja \cup \{Potomek\}$ 
16:   end while
17:    $Populacja \leftarrow NPopulacja$ 
18:    $CurrentBest \leftarrow ZnajdzNajlepszego(Populacja)$ 
19:   if  $CurrentBest.cost < Best.cost$  then
20:      $Best \leftarrow CurrentBest$ ;  $Stagnacja \leftarrow 0$ 
21:   else
22:      $Stagnacja \leftarrow Stagnacja + 1$ 
23:   end if
24:    $g \leftarrow g + 1$ 
25: end while
26: return  $Best$ 
```

C. Teoretyczna granica błędu względnego

Dopuszczalny błąd względny algorytmu skaluje się zgodnie z funkcją $O(24 \ln(n))$ i w uproszczeniu wynosi:

$$\delta = \begin{cases} 0\% & \text{dla } n < 25 \\ 50\% & \text{dla } 24 < n < 74 \\ 100\% & \text{dla } 75 < n < 449 \\ 150\% & \text{dla } 450 < n < 2500 \end{cases} \quad (1)$$

D. Konfiguracja eksperymentu

Parametry sterujące procesem oraz ścieżki do plików z danymi są definiowane w zewnętrznym pliku `config.txt`. Poniżej przedstawiono przykładową konfigurację pliku:

```
GRAPH_PATH = data/tsplib/tsp/pcb442.tsp
TIME_LIMIT_SECONDS = 60
REPETITIONS = 5
USE_INITIAL_UB = true
SAVE_TO_CSV = true
OPTIMAL_TOUR_COST = 50778
POPULATION_SIZE = 150.0
MUTATION_RATE = 0.05
GENERATIONS = 2000
CROSSOVER_METHOD = OX
MUTATION_METHOD = invert
```

E. Objaśnienie i rola parametrów sterujących algorytmu

W dalszej części stosowane będą następujące skróty oraz definicje formalne:

- **P_{size} (Rozmiar populacji):** Określa stałą liczbę osobników utrzymywanych jednocześnie w pamięci programu w trakcie trwania pojedynczego pokolenia. Parametr ten decyduje o szerokości frontu przeszukiwania przestrzeni stanów dopuszczalnych.
- **P_m (Współczynnik mutacji):** Prawdopodobieństwo poddania nowo wygenerowanego potomka modyfikacji genotypu. Współczynnik wyrażany jest w skali procentowej od 0.0 do 1.0. Jego głównym zadaniem jest wprowadzanie do populacji zupełnie nowych krawędzi (losowych zaburzeń), które mogły zostać bezpowrotnie utracone w procesie selekcji turniejowej.
- **Gen (Maksymalna liczba pokoleń):** Parametr określający limit iteracji pętli głównej algorytmu. Definiuje on maksymalny czas życia układu ewolucyjnego, w którym populacja ma możliwość adaptacji i selektywnego filtrowania najlepszych mikro-ścieżek.

F. Warunki zakończenia algorytmu

Algorytm Genetyczny kończy pracę gdy spełniony zostanie co najmniej jeden z poniższych warunków:

- **Limit czasu:** Przekroczenie maksymalnego czasu obliczeń wyznaczonego parametrem (domyślnie 900 sekund).
- **Osiągnięcie optimum:** Znalezienie rozwiązania o koszcie równym lub mniejszym od optimum globalnego.
- **Stagnacja populacji:** Brak poprawy najlepszego rozwiązania przez 20% całkowitej liczby pokoleń.

G. Fazy badawcze

W celu wyłonienia optymalnych parametrów i weryfikacji postawionych hipotez, zastosowano wielofazową procedurę testową. Każdy zestaw parametrów został zaprojektowany tak, aby izolować i badać wpływ konkretnych zmiennych konfiguracyjnych algorytmu.

- **Faza 1 (Identyfikacja operatora mutacji):** Przy parametrach bazowych ($P_{size} = 100$, $P_m = 0.10$, $Gen = 2000$) poddano weryfikacji metody *Swap*, *Insert* i *Invert*.
- **Faza 2 (Strojenie rozmiaru populacji i współczynnika mutacji):** Przy ustalonym operatorze *Swap* badano dynamikę algorytmu w funkcji rosnącej populacji ($P_{size} \in \{50, 100, 200\}$) oraz zmiennego prawdopodobieństwa mutacji ($P_m \in \{0.01, 0.05, 0.10\}$) zachowując stałą liczbę pokoleń ($Gen = 2000$).
- **Faza 3 (Wpływ głębokiej ewolucji - Invert):** Zastosowano progresywnie zwiększane ramy obliczeniowe dla metody *Invert*. Uogólnianie populacji i zbieżność poprzez znaczny wzrost liczby pokoleń (do $Gen = 10000$) przy jednoczesnym skalowaniu rozmiaru populacji (do $P_{size} = 300$).
- **Faza 4 (Pełna optymalizacja dla trudnych instancji - Insert):** Ekstremalne scenariusze obciążeniowe, podnosząc maksymalną liczbę pokoleń (do $Gen = 20000$) przy populacji rzędu $P_{size} = 500$ dla operatora *Insert*.

Szczegółowe zestawienia wykorzystanych kombinacji parametrów sterujących dla poszczególnych przebiegów optymalizacyjnych zaprezentowano w tabelach od I do IV.

Tabela I
FAZA 1: IDENTYFIKACJA OPERATORA MUTACJI

Lp.	P_{size}	P_m	Gen	Mutacja	Krzyżowanie
1	100	0.10	2000	Swap	OX
2	100	0.10	2000	Invert	OX
3	100	0.10	2000	Insert	OX

Tabela II
FAZA 2: STROJENIE ROZMIARU POPULACJI I WSPÓŁCZYNNIKA MUTACJI

Lp.	P_{size}	P_m	Gen	Mutacja	Krzyżowanie
1	50	0.05	2000	Swap	OX
2	100	0.05	2000	Swap	OX
3	200	0.05	2000	Swap	OX
4	100	0.01	2000	Swap	OX
5	100	0.10	2000	Swap	OX

Tabela III
FAZA 3: WPŁYW GŁĘBOKIEJ EWOLUCJI (INVERT)

Lp.	P_{size}	P_m	Gen	Mutacja	Krzyżowanie
1	100	0.10	2000	Invert	OX
2	200	0.10	5000	Invert	OX
3	300	0.15	10000	Invert	OX

Tabela IV
FAZA 4: PEŁNA OPTIMALIZACJA (INSERT)

Lp.	P_{size}	P_m	Gen	Mutacja	Krzyżowanie
1	200	0.10	5000	Insert	OX
2	300	0.15	10000	Insert	OX
3	500	0.15	20000	Insert	OX

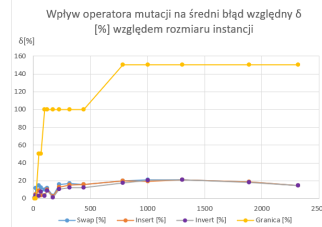
III. WYNIKI BADAŃ

A. Wpływ operatora mutacji

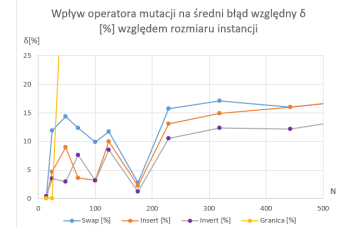
W tabeli V zestawiono błędy względne uzyskane przy stałych parametrach ($P_{size} = 100$, $P_m = 0.05$, $Gen = 1000$) dla trzech zaimplementowanych operatorów mutacji.

Tabela V
WPŁYW OPERATORA MUTACJI NA ŚREDNI BŁĄD WZGLĘDNY δ [%]

Instancja	Swap [%]	Insert [%]	Invert [%]
burma14	0.4	0.0	0.4
gr24	11.9	4.7	3.5
gr48	14.4	9.0	3.0
st70	12.4	3.6	7.6
kroA100	9.9	3.2	3.1
pr124	11.7	10.0	8.5
si175	2.8	2.2	1.2
gr229	15.7	13.1	10.5
lin318	17.1	14.9	12.4
pcb442	16.0	16.0	12.2
rat783	19.7	19.6	17.6
pr1002	21.1	19.2	20.7
rl1304	21.1	21.1	20.9
rl1889	18.4	18.4	18.2
u2319	14.8	14.8	14.3



Rysunek 1. Wpływ operatorów mutacji na średni błąd względny δ w funkcji rozmiaru instancji N .



Rysunek 2. Zbliżenie charakterystyki z Rys. 1 dla mniejszych instancji ($N \leq 500$).

B. Wpływ rozmiaru populacji i współczynnika mutacji P_m

Tabela VI pokazuje wyniki strojenia parametrów dla instancji referencyjnej gr229.

Tabela VI
ANALIZA PARAMETRYCZNA BŁĘDŲ δ I CZASU DLA INSTANCJI REFERENCYJNEJ

P_{size}	P_m [%]	Mutacja	Czas [us]	Błąd [%]
50	5.0	Swap	$2.7 \cdot 10^5$	16.6
100	1.0	Swap	$6.7 \cdot 10^5$	16.1
100	5.0	Swap	$5.9 \cdot 10^5$	15.7
100	10.0	Invert	$2.0 \cdot 10^6$	10.5
200	10.0	Invert	$1.0 \cdot 10^7$	4.3
200	10.0	Insert	$1.1 \cdot 10^7$	8.1

C. Wpływ liczby pokoleń

W tabeli VII pokazano zmianę jakości wraz ze wzrostem liczby pokoleń dla instancji gr229. W tej serii pomiarowej równolegle rosły także P_{size} i P_m , zgodnie z procedurą strojenia parametrów.

Tabela VII
WPŁYW LICZBY POKOLEŃ NA BŁĄD I CZAS (GR229, MUTACJA INVERT)

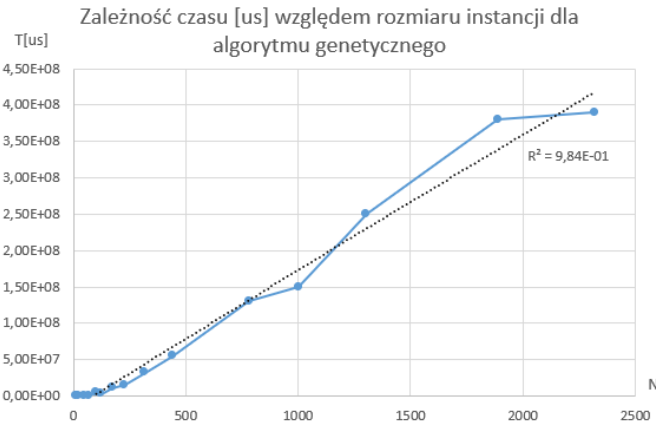
Pokolenia	P_{size}	P_m [%]	Czas [us]	Błąd [%]
2000	100	10.0	$2.0 \cdot 10^6$	10.5
5000	200	10.0	$1.0 \cdot 10^7$	4.3
10000	300	15.0	$1.5 \cdot 10^7$	2.8

D. Skalowalność, błąd względny i granica $O(24 \ln(n))$

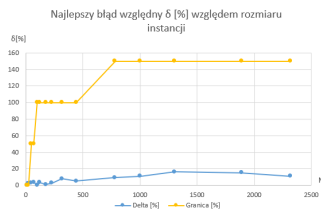
Tabela VIII przedstawia najlepsze wyniki dla każdej instancji, wraz z zestawem parametrów GA.

Tabela VIII
NAJLEPSZE WYNIKI GA DLA BADANYCH PRZEDZIAŁÓW Z PARAMETRAMI

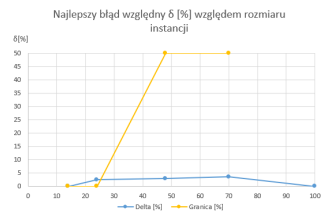
Inst.	N	P_{size}	Gen	Mutacja	P_m [%]	Czas [us]	δ [%]	Granica [%]
burma14	14	100	2000	insert	10	$2.3 \cdot 10^3$	0.0	0.0
gr24	24	200	5000	insert	10	$3.5 \cdot 10^5$	2.5	0.0
gr48	48	100	2000	invert	10	$2.6 \cdot 10^5$	3.0	50.0
st70	70	100	2000	insert	10	$6.9 \cdot 10^5$	3.6	50.0
kroA100	100	300	10000	invert	15	$4.5 \cdot 10^6$	0.0	100.0
pr124	124	200	5000	invert	10	$2.8 \cdot 10^6$	3.4	100.0
si175	175	300	10000	invert	15	$1.1 \cdot 10^7$	1.1	100.0
gr229	229	300	10000	invert	15	$1.5 \cdot 10^7$	2.8	100.0
lin318	318	300	10000	invert	15	$3.2 \cdot 10^7$	7.8	100.0
pcb442	442	300	10000	invert	15	$5.5 \cdot 10^7$	5.2	100.0
rat783	783	300	10000	invert	15	$1.3 \cdot 10^8$	9.3	150.0
pr1002	1002	300	10000	invert	15	$1.5 \cdot 10^8$	11.0	150.0
rl1304	1304	300	10000	invert	15	$2.5 \cdot 10^8$	16.3	150.0
rl1889	1889	300	10000	invert	15	$3.8 \cdot 10^8$	15.2	150.0
u2319	2319	300	10000	invert	15	$3.9 \cdot 10^8$	11.1	150.0



Rysunek 3. Zależność czasu obliczeń od rozmiaru instancji wraz z wielomianową linią trendu $O(N^2)$.



Rysunek 4. Najlepszy uzyskany błąd względny δ w zestawieniu z dopuszczalną granicą teoretyczną.



Rysunek 5. Zbliżenie charakterystyki z Rys. 4 uwidaczniające zachowanie dla początkowych wartości $N \leq 100$.

IV. WNIOSKI

A. Weryfikacja hipotez badawczych

Na podstawie przeprowadzonych eksperymentów i analizy danych zawartych w Sekcji III, sformułowano następujące wnioski dotyczące postawionych hipotez:

- **Weryfikacja H1 (Skalowalność):** Hipoteza została **potwierdzona częściowo**. Dane z tabeli VIII pokazują wyraźny, ponadliniowy wzrost czasu wraz z rozmiarem instancji oraz możliwość rozwiązywania bardzo dużych problemów w akceptowalnym czasie (dla u2319: $3.9 \cdot 10^8$ us, tj. ok. 390 s < 900 s). Jednocześnie pomiary były wykonywane dla różnych konfiguracji parametrów (P_{size} , P_m , Gen), dlatego nie stanowią czystego eksperymentu do ścisłego dopasowania wykładnika złożoności asymptotycznej.
- **Weryfikacja H2 (Wpływ rozmiaru populacji):** Hipoteza została **potwierdzona**. W tabeli VI zwiększanie rozmiaru populacji poprawia jakość rozwiązań, ale podnosi koszt obliczeń. Przykładowo dla mutacji Invert i $P_m = 10\%$ przejście z $P_{size} = 100$ do $P_{size} = 200$ obniża błąd z 10.5% do 4.3%, lecz wydłuża czas z $2.0 \cdot 10^6$ us do $1.0 \cdot 10^7$ us.
- **Weryfikacja H3 (Wpływ prawdopodobieństwa mutacji):** Hipoteza została **wstępnie potwierdzona**. Dla porównywalnych ustawień z tabeli VI wzrost P_m z 1% do 5% (mutacja Swap, $P_{size} = 100$) zmniejsza błąd z 16.1% do 15.7%. Kierunek zmian jest zgodny z oczekiwaniem, jednak dla pełnej izolacji wpływu P_m potrzebna byłaby szersza seria pomiarów przy niezmienionym operatorze i pozostałych parametrach.
- **Weryfikacja H4 (Wpływ liczby pokoleń):** Hipoteza została **potwierdzona**. Z tabeli VII wynika, że zwiększanie liczby pokoleń poprawia jakość rozwiązania (10.5% → 4.3% → 2.8%), ale kosztem wzrostu czasu ($2.0 \cdot 10^6$ us → $1.0 \cdot 10^7$ us → $1.5 \cdot 10^7$ us). Widoczny jest efekt malejących korzyści: poprawa między 5000 a 10000 pokoleń jest mniejsza niż między 2000 a 5000.
- **Weryfikacja H5 (Skuteczność operatora Invert):** Hipoteza została **w dużej mierze potwierdzona**. W tabeli V operator *Invert* osiąga najniższy błąd dla większości instancji (12 z 15). Wyjątki stanowią m.in. burma14, st70 oraz pr1002, gdzie lepsze wyniki uzyskał operator *Insert*. Oznacza to, że *Invert* jest operatorem dominującym, lecz nie uniwersalnie najlepszym.

B. Weryfikacja założenia o granicy błędu względnego

Głównym celem projektowym postawionym przed algorytmem genetycznym było znalezienie optymalnych parametrów sterujących, aby błąd względny δ znalezionych rozwiązań nie przekroczył granic wyznaczonych przez uproszczoną funkcję $O(24 \ln(n))$. Analiza Tabeli VIII oraz wykresów (Rys. 4 i Rys. 5) pozwala stwierdzić, że założenie to zostało zrealizowane dla niemal wszystkich przedziałów, z jednym odstępstwem w najniższym zakresie:

- **Dla $N < 25$ (limit 0%):** Kryterium to okazało się najbardziej rygorystyczne ze względu na swoją bezwzględność. O ile dla instancji *burma14* algorytm poprawnie znalazł globalne optimum (błąd 0,0%), o tyle dla instancji *gr24* minimalny znaleziony błąd wyniósł 2,5%.
- **Dla $24 < N < 74$ (limit 50%):** Uzyskane błędy dla instancji *gr48* (3,0%) oraz *st70* (3,6%) leżą poniżej dopuszczalnego progu 50%.
- **Dla $75 < N < 449$ (limit 100%):** Algorytm utrzymał błąd na poziomie kilku procent (2,8% dla *gr229*, 5,2% dla *pcb442*), podczas gdy dopuszczalny limit wynosił 100%.
- **Dla $450 < N < 2500$ (limit 150%):** Dla instancji o gigantycznej przestrzeni poszukiwań (*u2319*) algorytm wygenerował trasę z błędem 11,1%, co stanowi wynik kilkunastokrotnie lepszy od wyznaczonej granicy 150%.

Podsumowując, zestawienie parametrów P_{size} , P_m oraz Gen pozwoliło na znaczne zdominowanie wymagań we wszystkich przedziałach powyżej 24 wierzchołków, udowadniając przydatność zaprojektowanej metaheurystyki.

Przypuszczalny brak zejścia do poziomu 0,0% dla instancji *gr24* to skutek losowego elementu algorytmu genetycznego, który łatwo ulega przedwczesnej zbieżności i stabilizuje się w silnych minimach lokalnych, różniących się od globalnego optimum.

C. Charakterystyka algorytmu

Algorytm Genetyczny charakteryzuje się bardzo dobrą skalowalnością, co czyni go idealnym wyborem dla dużych instancji TSP, dla których metody dokładne stają się bezużyteczne z uwagi na wykładniczy wzrost czasu obliczeń. GA nadaje się do zastosowań inżynierskich i logistycznych, gdzie wymagane jest znalezienie bardzo dobrego rozwiązania w rozsądnym czasie.

Algorytm ten nie nadaje się jednak do systemów wymagających bezwzględnej gwarancji znalezienia matematycznego optimum globalnego. Ponadto, ze względu na dużą liczbę stochastycznych parametrów sterujących, wymaga on czasochłonnego strojenia do konkretnego typu instancji, co czyni go mało elastycznym.

LITERATURA

- [1] *VLSI TSP Data*, University of Waterloo. [Online]. Dostępne: <https://www.math.uwaterloo.ca/tsp/vlsi/index.html>
- [2] Mecalux, *Problem komiwojazera w logistyce*. [Online]. Dostępne: <https://www.mecalux.pl/blog/problem-komiwojazera>
- [3] W. Kwedło, *Algorytmy Ewolucyjne - Wykład*, Politechnika Białostocka. [Online]. Dostępne: <https://www.aragorn.wi.pb.edu.pl/~wkwedlo/EA5.pdf>
- [4] *Algorytmy Genetyczne - Materiały dydaktyczne*, Akademia Górniczo-Hutnicza. [Online]. Dostępne: <http://www.pi.zarz.agh.edu.pl/swd/wyklady/AG%20razem.pdf>